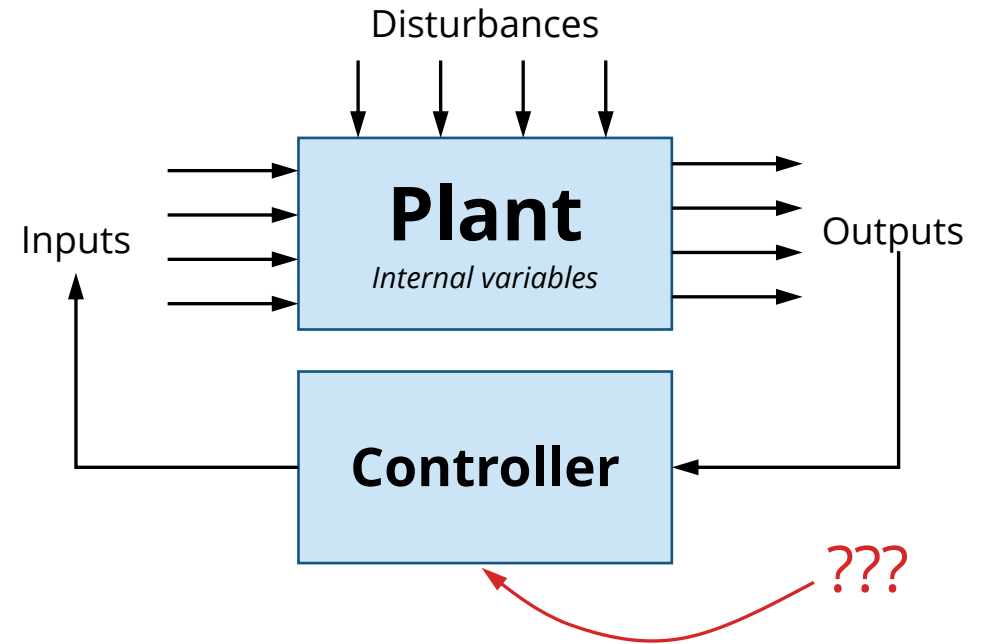




AA 513 Multivariable Control

Lecture 04 // Week 4

Instructor: Prof. Karen Leung

Announcements

- Self-assessment due Thursday
- Extra credit for Ed Discussion participation
 - Great discussion so far!
- Course website contains some materials on how to navigate research papers
 - + short in-class demo
- Guest lecture next week by Dr Liam Kruse. Topic: Decision-making under uncertainty / Markov Decision Processes
 - Paradigm for reinforcement learning

How to search for interesting research papers

- General approach: Find research groups that you find interesting and look at their latest publications
- Finding research groups / areas of interests:
 - Check out seminars
 - Check out *workshops* at conferences.
- Accessing papers
 - ArXiv
 - Use UW account login
 - Go to lab/author website
- Useful links
 - [Connected papers](#)
 - Google Scholar

Last time

- Equilibria
- Stability
- Lyapunov functions – certificate for stability
 - Find V that satisfies 3 conditions
- Control Lyapunov Functions
 - Find V such that there exists a control that the three conditions are met
- Set invariance – If starting inside the set, system will remain inside of set
 - Control invariance - there exists a control such that...
 - Lyapunov level sets are (control) invariant sets
- Brief into to Control Barrier functions

This week

- Finish off Control Barrier Functions
- Sequential decision-making
 - <https://uw-ctrl.github.io/lmc-book/lectures/sequential-dm.html>

Exercises

- Ex4 Equilibria
- Ex5 Lyapunov function
- Ex6 Control Barrier Functions and safety filters

From Lyapunov to Barrier functions

- **Lyapunov Functions:** Mechanism to certify convergence to a state
- ***Control* Lyapunov Functions:** Mechanism to certify and synthesize controls to converge to a state

Convergence to state → Stay inside set

- **Barrier Functions:** Mechanism to certify a system's ability to keep within a set
- ***Control* Barrier Functions** provide the mechanism to certify and synthesize controls to keep a system within a set

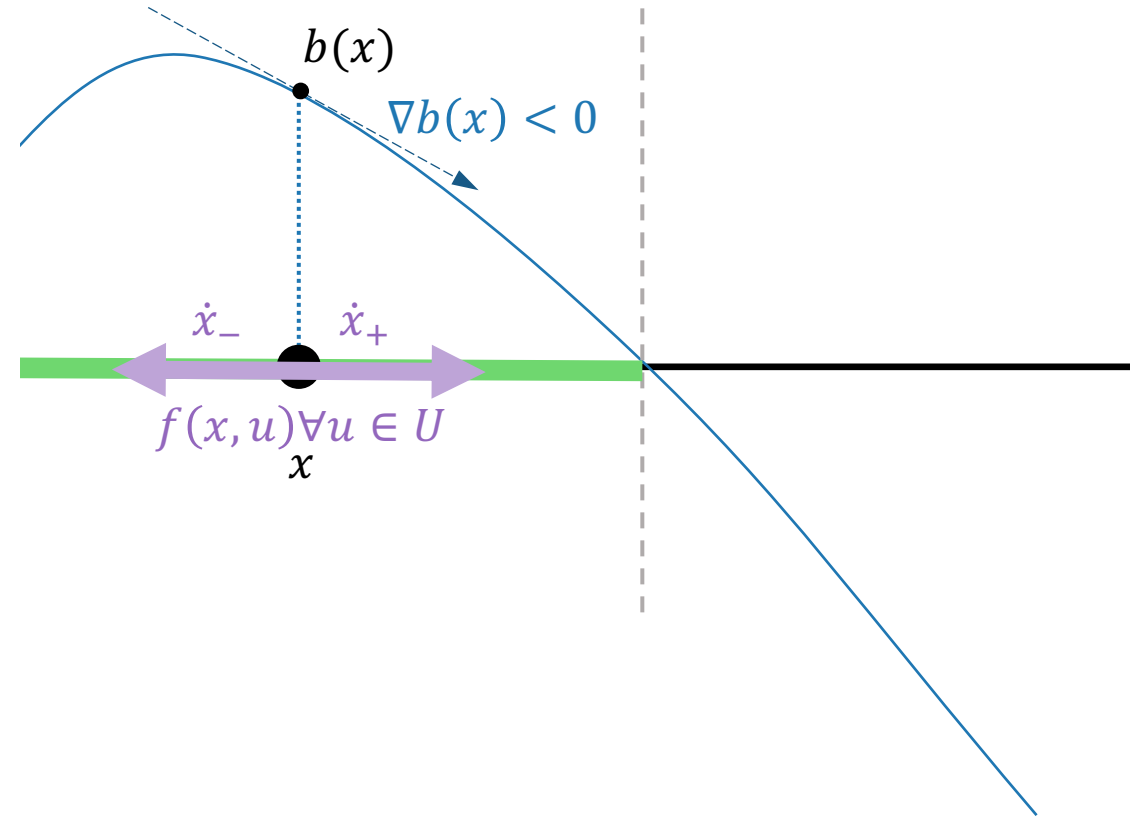
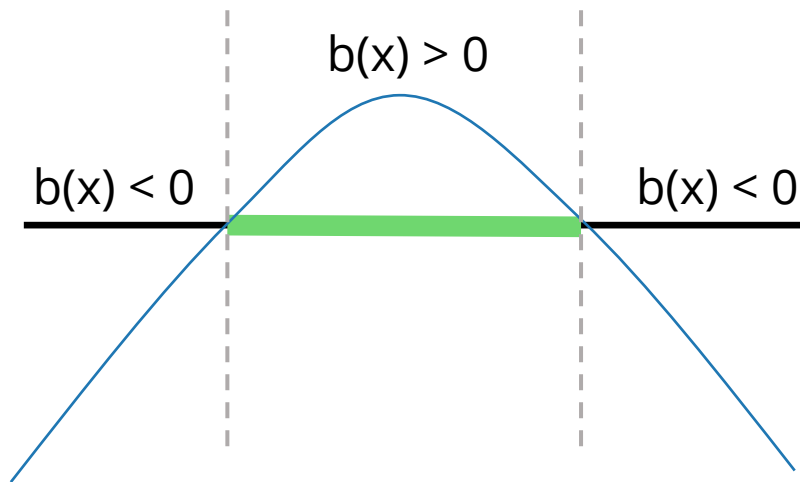
Control Barrier Function

- Let $S = \{x \in X \mid b(x) \geq 0\}$
- Let $\alpha: R \rightarrow R$ be an extended class K function
 - $\alpha(0) = 0$
 - Strictly increasing
 - E.g., $\alpha(x) = ax$
- Then b is a valid CBF for the given α if $\forall x \in X, \exists u \in U$ such that $\nabla b(x)^T f(x, u) \geq -\alpha(b(x))$
- Differently put,

$$\max_{u \in U} \underbrace{\nabla b(x)^T f(x, u)}_{\dot{b}(x)} \geq -\alpha(b(x)), \forall x \in X$$

Control Barrier Functions: Visualization

$$\nabla b(x)^T f(x, u) \geq -\alpha(b(x))$$



$\nabla b(x)^T \dot{x}_-$ ok?

$\nabla b(x)^T \dot{x}_+$ ok?

 X domain
 Constraint set (keep inside)
 Control Barrier function $b(x)$

The CBF inequality constraint

$$\underbrace{\nabla b(x)^T f(x, u)}_{\dot{b}(x)} \geq -\alpha(b(x))$$

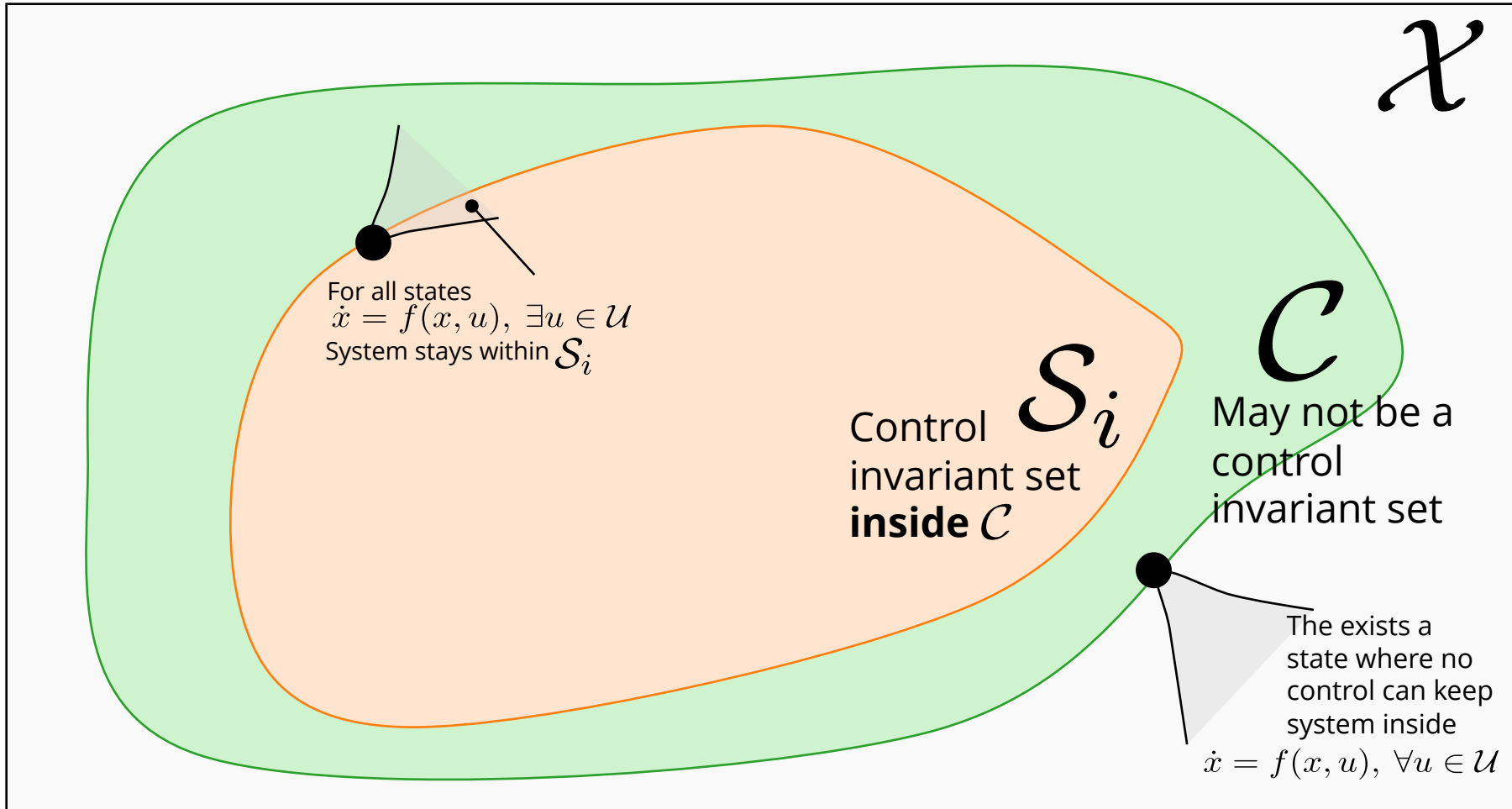
$$\alpha(x) = ax, a \text{ small}$$

$$\alpha(x) = ax, a \text{ large}$$

Deep inside S ,
 $b(x)$ large

Close to boundary of S ,
 $b(x)$ small

Visualization of sets



$\mathcal{X}$ Domain

$\mathcal{C}$ Constraint set.
Must stay inside of this at all times.

$\mathcal{S}_i$ A control invariant set contained within $\mathcal{C}$

Possibly multiple $\mathcal{S}_i$

$\mathcal{S}_1, \mathcal{S}_2, \dots$

Constraint set vs “Safe” set

- Constraint set may not be a control invariant set
- E.g., Double integrator 2D dynamics
 - State: $[x, y, \dot{x}, \dot{y}]$
 - Control: $[\ddot{x}, \ddot{y}] \in [-a_{\max}, a_{\max}] \times [-a_{\max}, a_{\max}]$, $a_{\max} < \infty$
 - Constraint set: $\mathcal{C} = \{(x, y) | x^2 + y^2 \leq r^2\}$
 - $x(t) = [r, 0, 1., 0] \in \mathcal{C}$ but $x(t + \epsilon) \notin \mathcal{C}$
- What can we do?
 - Choose $S \subset \mathcal{C}$ where S considers positions AND velocity
 - Intuitively, shrink $\mathcal{C}$ by the stopping distance
 - But typically non-trivial for general nonlinear systems

Some caveats for CBFs

- Relative degree: When $\nabla b(x)^T g(x) = 0$, for $\dot{x} = f_0(x) + g(x)u$
- Finding the *largest* control invariant set within $\mathcal{C}$
- Need to find a valid α
- May conflict with other objectives or constraints
- There is a discrete-time version

CBF safety filter

- We can use CBFs to *filter*, or adjust, the desired control to ensure safety (and stability) conditions are satisfied.
- For current state x , and desired control input $u^{desired}$ (e.g., from high-level planner)

$$\begin{aligned} & \min_u ||u - u^{desired}||_2^2 \\ \text{s. t. } & \nabla b(x)^T (f_0(x) + g(x)u) \geq -\alpha(b(x)) \\ & |u| \leq u^{limit} \end{aligned}$$

This is a quadratic program! Convex! Can use CVXPY!

CBF-CLF filter

- We can add a CLF constraint as well!

$$\begin{aligned} & \min_u ||u - u^{desired} ||_2^2 \\ \text{s.t. } & \nabla b(x)^T (f_0(x) + g(x)u) + \alpha_{CBF}(b(x)) \geq 0 \\ & \nabla V(x)^T (f_0(x) + g(x)u) + \alpha_{CLF}V(x) \leq 0 \\ & |u| \leq u^{limit} \end{aligned}$$

Problems?

Add slack variables to relax the constraints

$$\begin{aligned} \min_u \quad & ||u - u^{desired}||_2^2 + \gamma_{CBF} \epsilon_{CBF}^2 + \gamma_{CLF} \epsilon_{CLF}^2 \\ \text{s.t.} \quad & \nabla b(x)^T (f_0(x) + g(x)u) + \alpha_{CBF}(b(x)) \geq -\epsilon_{CBF} \\ & \nabla V(x)^T (f_0(x) + g(x)u) + \alpha_{CLF}V(x) \leq \epsilon_{CLF} \\ & |u| \leq u^{limit} \\ & \epsilon_{CLF} \geq 0, \epsilon_{CBF} \geq 0 \end{aligned}$$

Code demo

Exercise 6

Sequential decision making (SDM)

- Decisions, synonymous with controls, or actions
- We want to make a sequence of decisions over a time horizon
 - E.g., sequence of control inputs to move system from point A to point B
- **Challenges:**
 - Need to reason about the *consequences* of decisions
 - *Immediate* consequences vs *future* consequences
 - Uncertainty in the future
 - Large state space
 - Knowing true value of the consequence

Group activity

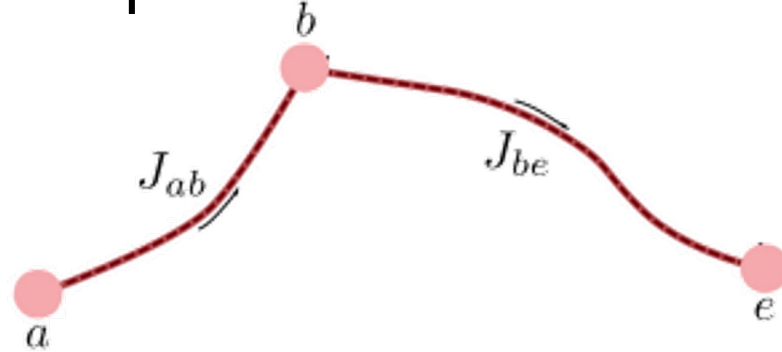
- Describe some real-world settings where you need to perform sequential decision making
 - Immediate consequence
 - Future consequence
 - States and decisions/controls
 - Potential sources of uncertainties, and potential ways to address them
- E.g., Chess
 - Taking opponent's pieces
 - Setting up board to be in a position to win
 - Location of pieces
 - Piece to move
 - What the opponent does on their move
 - Could assume opponent is optimal

Principle of Optimality

- An optimal solution contains within it optimal solutions to its subproblems
- Given an optimal solution to a SDM problem, then any tail portion of the solution is also optimal to that sub SDM problem
- When following an optimal solution, the remainder of the solution must also be optimal from that state onwards.
- If the problem is solved optimally, then regardless of past decisions, future decisions must be optimal for the current state.

Principle of optimality: Setup

- Suppose optimal path for a multi-stage decision-making problem is

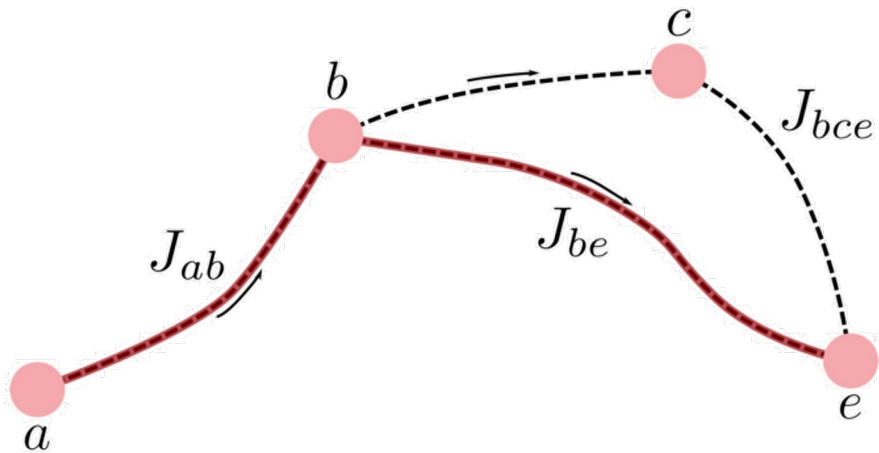


- First decision yields segment $a - b$ with cost J_{ab}
- Remaining decisions yield segment $b - e$ with cost J_{be}
- Optimal cost is then $J_{ae}^* = J_{ab} + J_{be}$
- **Assumption:** Cost has *additive* structure

Adapted from Stanford AA 203 course notes

Principle of optimality

- **Claim:** If $a - b - e$ is the optimal path from a to e , then $b - e$ is an optimal path from b to e
- **Proof:** By contradiction



Adapted from Stanford AA 203 course notes

Implications of the PoO

- To find optimal decision sequence, we consider solving tail subproblems.
- Then consider solving the tail subproblems of those subproblems, and so forth.
- Start with the smallest tail subproblem and work our way backwards
- Can ignore the past, just concerned with optimizing the future from the current situation
 - Need to solve for *all* tail subproblems



Dynamic programming

- Algorithmic technique for solving complex problems by breaking them down into simpler subproblems
- **Core idea:** Solve each subproblem only once and store the solution
- **Benefit:** Prevents redundant computations when subproblem is encountered again
- A common technique to solve many problems, not just control problems

Dynamic programming examples

-  Dividing text of paragraph into lines of approximately equal length—Knuth–Plass line-breaking algorithm used dynamic programming
-  Route optimization, traveling salesman problem, orienteering problem. Useful for delivery route planning, surveillance missions, transportation routing, etc
-  Sequence alignment: Comparing DNA or protein sequences to find similarities (e.g., Needleman–Wunsch or Smith–Waterman algorithms).

Examples

- Graph example
- Grid world example

To start simple, consider discrete **time**,
discrete **state**, discrete **control**

Principle of optimality: Control setting

- **System** $x_{t+1} = f(x_t, u_t, t), t = 0, \dots, N - 1$, **control constraints** $u_t \in U(x_t)$.
- **Cost** $J(x_0, u_{0:N-1}) = g_N(x_N) + \sum_{t=0}^{N-1} g(x_t, u_t, t)$
- Let $\{u_0^*, u_1^*, \dots, u_{N-1}^*\}$ be an *optimal* control sequence starting at state x_0 , with corresponding state sequence $\{x_0^*, x_1^*, \dots, x_N^*\}$
- Consider subproblem
- What is the optimal control sequence starting at x_k^* ?
- What is the cost that it minimizes?

Cost-to-go, a.k.a, the Value Function

- Given an optimal policy $\mathbf{u}_t^* = \pi^*(\mathbf{x}_t, t)$, starting at $\mathbf{x}_0$
- Optimal cost is $J^*(\mathbf{x}_0, 0) = g_N(\mathbf{x}_N) + \sum_{t=0}^{N-1} g(\mathbf{x}_t, \pi^*(\mathbf{x}_t, t), t)$
- At $(\mathbf{x}_t, t)$ the optimal cost is

$$J^*(\mathbf{x}_t, t) = g_N(\mathbf{x}_N) + \sum_{k=t}^{N-1} g(\mathbf{x}_k, \pi^*(\mathbf{x}_k, k), k)$$

- Alternatively,

$$J^*(\mathbf{x}_t, t) = g(\mathbf{x}_t, t) + g_N(\mathbf{x}_N) + \sum_{k=t+1}^{N-1} g(\mathbf{x}_k, \pi^*(\mathbf{x}_k, k), k)$$

$$J^*(\mathbf{x}_t, t) = g(\mathbf{x}_t, t) + J^*(\mathbf{x}_{t+1}, t + 1)$$

Cost-to-go, a.k.a, the Value Function

- Cost-to-go, also referred to as the **Value Function**
 - The future total cost at $(\mathbf{x}_t, t)$ if following policy π

$$V_{\pi}(\mathbf{x}_t, t) = g_N(\mathbf{x}_N) + \sum_{k=t}^{N-1} g(\mathbf{x}_k, \pi(\mathbf{x}_k, k), k)$$

$$V_{\pi}(\mathbf{x}_t, t) = g(\mathbf{x}_t, \pi(\mathbf{x}_t, t), t) + V_{\pi}(\mathbf{x}_{t+1}, t + 1)$$

- Optimal value function corresponds to following optimal policy

Breaking it down further

$$V_{\pi}(\mathbf{x}_t, t) = g_N(\mathbf{x}_N) + \sum_{k=t}^{N-1} g(\mathbf{x}_k, \pi(\mathbf{x}_k, k), k)$$

- $(\mathbf{x}_0, t = 0)$ $V_{\pi}(\mathbf{x}_0, 0) = g_N(\mathbf{x}_N) + \sum_{k=0}^{N-1} g(\mathbf{x}_k, \pi(\mathbf{x}_k, k), k)$
 $= g(\mathbf{x}_0, \pi(\mathbf{x}_0, 0), 0) + V_{\pi}(\mathbf{x}_1, 1)$
- $(\mathbf{x}_1, t = 1)$ $V_{\pi}(\mathbf{x}_1, 1) = g(\mathbf{x}_1, \pi(\mathbf{x}_1, 1), 1) + V_{\pi}(\mathbf{x}_2, 2)$
- $(\mathbf{x}_2, t = 2)$ $V_{\pi}(\mathbf{x}_2, 2) = g(\mathbf{x}_2, \pi(\mathbf{x}_2, 2), 2) + V_{\pi}(\mathbf{x}_3, 3)$
- ...
- $(\mathbf{x}_{N-1}, t = N - 1)$ $V_{\pi}(\mathbf{x}_{N-1}, N - 1) = g(\mathbf{x}_{N-1}, \pi(\mathbf{x}_{N-1}, N - 1), N - 1) + V_{\pi}(\mathbf{x}_N, N)$
- $(\mathbf{x}_N, t = N)$. What is the cost at final state at final time?
 - $V_{\pi}(\mathbf{x}_N, N) =$

What have we just done?

- Recursively broken up the total cost $J(\mathbf{x}_0, 0)$ into smaller subparts
 - $V_\pi(\mathbf{x}_t, t)$ = current cost $(\mathbf{x}_t, \mathbf{u}_t = \pi(\mathbf{x}_t, t), t)$ + cost-to-go at $\mathbf{x}_{t+1}$
 - $V_\pi(\mathbf{x}_{t+1}, t + 1)$ = current cost $(\mathbf{x}_{t+1}, \mathbf{u}_{t+1} = \pi(\mathbf{x}_{t+1}, t + 1), t + 1)$ + cost-to-go at $\mathbf{x}_{t+2}$
- Until at the last time step
 - $V_\pi(\mathbf{x}_N, N) = g_N(\mathbf{x}_N)$ = terminal cost which is known
- Then we can sweep *backward* in time to solve for all the $V_\pi(\mathbf{x}_t, t)$

This is ***dynamic programming!***

The value function

- The value function describes the ***future accumulated cost***
 - Depends on the policy taken
- Given a policy π : $V_{\pi}(\mathbf{x}_t, t) = g_N(\mathbf{x}_N) + \sum_{k=t}^{N-1} g(\mathbf{x}_k, \pi(\mathbf{x}_k, k), k)$
- Denote optimal policy with π^*
 - Optimal value: $V^*(\mathbf{x}_t, t) := V_{\pi^*}(\mathbf{x}_t, t) = g_N(\mathbf{x}_N) + \sum_{k=t}^{N-1} g(\mathbf{x}_k, \pi^*(\mathbf{x}_k, k), k)$
- **Interpretation:** Measures the “value”, “cost-to-go”, or “goodness/badness” of being in state $\mathbf{x}$ at timestep t

Use dynamic programming to solve for optimal policy and value

- How to find the optimal policy? π^* is to be determined

$$V^*(\mathbf{x}_N, N) = g_N(\mathbf{x}_N)$$

- At $(\mathbf{x}_{N-1}, N-1)$, pick the control that minimizes sum of immediate cost + value at subsequent state

$$V^*(\mathbf{x}_{N-1}, N-1) = \min_{u_{N-1} \in U} g(\mathbf{x}_{N-1}, u_{N-1}, N-1) + V^*(\mathbf{x}_N, N)$$

$$= \min_{u_{N-1} \in U} g(\mathbf{x}_{N-1}, u_{N-1}, N-1) + V^*(f(\mathbf{x}_{N-1}, u_{N-1}, N-1), N)$$

Use dynamic programming to solve for optimal policy and value

$$V^*(\mathbf{x}_{N-1}, N-1) = \min_{u_{N-1} \in U} g(\mathbf{x}_{N-1}, u_{N-1}, N-1) + V^*(f(\mathbf{x}_{N-1}, u_{N-1}, N-1), N)$$
$$\pi^*(\mathbf{x}_{N-1}, N-1) = \operatorname{argmin}_{u_{N-1} \in U} g(\mathbf{x}_{N-1}, u_{N-1}, N-1) + V^*(f(\mathbf{x}_{N-1}, u_{N-1}, N-1), N)$$

- Pick best control that minimizes immediate cost + cost-to-go at next state
- Repeat process, stepping backward in time

Consider discrete
time, discrete **state**,
discrete **control**

$$V^*(\mathbf{x}_t, t) = \min_{u_t \in U} g(\mathbf{x}_t, u_t, t) + V^*(f(\mathbf{x}_t, u_t, t), t+1)$$

$$\pi^*(\mathbf{x}_t, t) = \operatorname{argmin}_{u_t \in U} g(\mathbf{x}_t, u_t, t) + V^*(f(\mathbf{x}_t, u_t, t), t+1)$$

Bellman equation (discrete time)

$$V^*(\mathbf{x}_t, t) = \min_{u_t \in U} g(\mathbf{x}_t, u_t, t) + V^*(f(\mathbf{x}_t, u_t, t), t + 1)$$

$$\pi^*(\mathbf{x}_t, t) = \operatorname{argmin}_{u_t \in U} g(\mathbf{x}_t, u_t, t) + V^*(f(\mathbf{x}_t, u_t, t), t + 1)$$

- If discrete state and control, store value and policy in a table and perform a look-up
- Computational challenges with very large state space

Bellman equation (discrete time)

$$V^*(\mathbf{x}_t, t) = \min_{u_t \in U} g(\mathbf{x}_t, u_t, t) + V^*(f(\mathbf{x}_t, u_t, t), t + 1)$$

$$\pi^*(\mathbf{x}_t, t) = \operatorname{argmin}_{u_t \in U} g(\mathbf{x}_t, u_t, t) + V^*(f(\mathbf{x}_t, u_t, t), t + 1)$$

- What about continuous state and control (still discrete time)?
- What about stochasticity?
- What about infinite horizon?
- What about continuous state and action?

Stochastic Bellman Equation

- Suppose there is uncertainty in the dynamics.
 - Deterministic: $\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t, t)$
 - Stochastic: $\mathbf{x}_{t+1} \sim p(\cdot | \mathbf{x}_t, \mathbf{u}_t, t)$ (A conditional probability)

- Deterministic Bellman:

$$V^*(\mathbf{x}_t, t) = \min_{\mathbf{u}_t \in U} g(\mathbf{x}_t, \mathbf{u}_t, t) + V^*(f(\mathbf{x}_t, \mathbf{u}_t, t), t + 1)$$

- Stochastic Bellman

$$V^*(\mathbf{x}_t, t) = \min_{\mathbf{u}_t \in U} \mathbb{E}_{\mathbf{x}_{t+1} \sim p(\cdot | \mathbf{x}_t, \mathbf{u}_t, t)} [g(\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1}, t) + V^*(\mathbf{x}_{t+1}, t + 1)]$$

$$V^*(\mathbf{x}_t, t) = \min_{\mathbf{u}_t \in U} \sum_{\mathbf{x}_{t+1} \in X} (g(\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1}, t) + V^*(\mathbf{x}_{t+1}, t + 1)) p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t, t)$$

Infinite horizon

- If infinite horizon, then dealing with infinite sum
 - Concern that the value will not converge / have finite value
- **Solution:** Add a **discount factor** γ to slightly devalue consequences in the future.

- **Deterministic:**

$$V^*(\mathbf{x}_t, t) = \min_{\mathbf{u}_t \in U} g(\mathbf{x}_t, \mathbf{u}_t, t) + \gamma V^*(f(\mathbf{x}_t, \mathbf{u}_t, t), t + 1)$$

- **Stochastic:**

$$V^*(\mathbf{x}_t, t) = \min_{\mathbf{u}_t \in U} \sum_{\mathbf{x}_{t+1} \in X} (g(\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1}, t) + \gamma V^*(\mathbf{x}_{t+1}, t + 1)) p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t, t)$$

Continuous-time, continuous state & control

- Consider cost $J(\mathbf{x}_0, 0) = g_N(\mathbf{x}(T)) + \int_0^T g(\mathbf{x}(t), \mathbf{u}(t), t) dt$
- Cast as discrete time problem
 - $g_{discrete}(\mathbf{x}_t, \mathbf{u}_t, t) = \int_t^{t+\epsilon} g(\mathbf{x}(t), \mathbf{u}(t), t) dt \approx \epsilon g(\mathbf{x}(t), \mathbf{u}(t), t)$, for small ϵ
- $V^*(\mathbf{x}(t), t) = \min_{\mathbf{u}_t \in U} \epsilon g(\mathbf{x}(t), \mathbf{u}(t), t) + V^*(\mathbf{x}(t + \epsilon), t + \epsilon)$
- $0 = \min_{\mathbf{u}_t \in U} \epsilon g(\mathbf{x}(t), \mathbf{u}(t), t) + V^*(\mathbf{x}(t + \epsilon), t + \epsilon) - V^*(\mathbf{x}(t), t)$
- $0 = \min_{\mathbf{u}_t \in U} g(\mathbf{x}(t), \mathbf{u}(t), t) + \frac{1}{\epsilon} (V^*(\mathbf{x}(t + \epsilon), t + \epsilon) - V^*(\mathbf{x}(t), t))$
- $0 = \lim_{\epsilon \rightarrow 0} \min_{\mathbf{u}_t \in U} g(\mathbf{x}(t), \mathbf{u}(t), t) + \frac{1}{\epsilon} (V^*(\mathbf{x}(t + \epsilon), t + \epsilon) - V^*(\mathbf{x}(t), t))$
- $0 = \min_{\mathbf{u}_t \in U} g(\mathbf{x}(t), \mathbf{u}(t), t) + \frac{\partial V}{\partial t}(\mathbf{x}(t), \mathbf{u}(t)) + \nabla V(\mathbf{x}(t), t)^T f(\mathbf{x}(t), \mathbf{u}(t), t)$

Hamilton-Jacobi-Bellman equation

$$0 = \frac{\partial V}{\partial t}(x, t) + \min_{u \in U} g(x, u, t) + \nabla V(x, t)^T f(x, u, t)$$

- A **partial differential equation**
- Generally hard to solve exactly
 - Can solve numerically for small-ish problems
- Under what problem settings with continuous state and control, can we solve it analytically?
 - Find out in Week 6 😊